AI Network Intrusion Detection System

Simple 3-Week Team Plan (5 People)

Purpose: Build a class project that uses machine learning to predict whether a network-traffic record is **normal (0)** or **an attack (1)**. We are comparing different AI methods on existing data - **not** collecting live traffic or making a production security tool.

How we work: Everyone works in their own folder, on their own schedule. You do **not** need to wait for another person's code. During Week 3, we put everyone's finished results together.

Who does what?

Person	Main job	What they hand in
1	Explore the dataset + train a simple Logistic Regression model	Dataset summary + baseline predictions
2	Train a **Random Forest** classifier	Model script + predictions
3	Train an **SVM** classifier	Model script + predictions
4	Train an **Isolation Forest** anomaly detector	Model script + predictions
5	Build a simple results dashboard and compare the models	Streamlit dashboard + comparison

Read this first (everyone)

1. Get the two **prepared** UNSW-NB15 CSV files from the official dataset page: `UNSW_NB15_training-set.csv` and `UNSW_NB15_testing-set.csv`. Do not use the much larger raw data files. The prepared training file has **175,341 rows** and the test file has **82,332 rows**.
2. Use Python and work only in your assigned folder. Install what *your own* script needs (usually `pandas` and `scikit-learn`; Person 5 also needs `streamlit`).
3. For training, use the **training CSV**. Use the **testing CSV** only for final predictions and evaluation. The `label` column is the answer: `0 = normal`, `1 = attack`.
4. **Never give the model the answer as an input.** Remove `id`, `label`, and `attack_cat` from the input features. If you need to turn text columns into numbers or scale values, fit those steps on training data only.
5. At the end, each model person saves one results CSV in the format below. No shared preprocessing script is required.

```
row_id,y_true,y_pred
0,0,0
1,1,1
2,1,0
```

- `row_id` is the **row position** in the original test CSV, starting at 0; do not rearrange test rows.
- `y_true` is the real `label` from the test CSV; `y_pred` is your model's `0` or `1` prediction.
- Submit **all 82,332 test rows**, in original order. The three rows above are just an example.
- Everyone's files can be compared because the rows and column names are identical.

Where your work goes

Keep the downloaded CSV files in a local `data/` folder. Put your own code in the folder named in your **Submit** line, and write your predictions into `results/p1.csv`, `results/p2.csv`, `results/p3.csv`, or `results/p4.csv`. **Do not edit anyone else's folder.** Use separate Git branches if you're sharing one repository.

Weekly check-in: Post only three things: **done / next / blocked**. If someone is behind, the others keep working; that person's prediction file can be added during Week 3.

Person 1 - Understand the Data + Baseline

In plain English: Explain what information the network dataset contains and make a basic prediction model. Your model is the starting point the others will try to beat.

Week 1

- ☐ Download both CSV files and open them with pandas.
- ☐ Count normal vs. attack rows; find any missing data; make **1-2 simple charts**.
- ☐ Write a short explanation (about one paragraph) of what the dataset is.

Week 2

- ☐ Train a **Logistic Regression** classifier using the training CSV.
- ☐ Convert categorical columns to numbers and scale numerical columns as needed (fit on training data only).
- ☐ Predict on the unchanged testing CSV and save `results/p1.csv`.

Week 3

- ☐ Record **accuracy, precision, recall, and F1-score**.
- ☐ Write 3-5 sentences: what worked, what was hard, and what the results mean.
- ☐ Confirm the script runs independently and upload your files.

Submit: `person1_baseline/train.py`, `person1_baseline/README.md`, 1-2 charts, and `results/p1.csv`.

Person 2 - Random Forest Model

In plain English: Train a model that makes decisions using many small decision trees. It is a common method for classifying data.

Week 1

- ☐ Download the same two CSVs and look at the column names.
- ☐ Create `person2_random_forest/train.py` and load the data.
- ☐ Prepare the input columns in your own script (exclude `id`, `label`, and `attack_cat`).

Week 2

- ☐ Train `RandomForestClassifier` from scikit-learn.
- ☐ Run predictions on the official testing CSV.
- ☐ Save `results/p2.csv` using the required three columns.

Week 3

- ☐ Record accuracy, precision, recall, and F1-score.
- ☐ Make **one** simple chart or table of your model's results.
- ☐ Write 3-5 sentences explaining your approach and findings.

Submit: `person2_random_forest/train.py`, `person2_random_forest/README.md`, results chart/table, and `results/p2.csv`.

Person 3 - SVM Model

In plain English: Train another classifier that learns a boundary between normal network activity and attacks. We want to compare it against Random Forest.

Week 1

- ☐ Download the same CSVs and set up `person3_svm/train.py`.
- ☐ Prepare the inputs; use one-hot encoding for text columns and scaling for numeric columns.
- ☐ Start with a **small sample of training rows** to check that the code works.

Week 2

- ☐ Train scikit-learn's `LinearSVC` (a faster, practical SVM choice for this dataset).
- ☐ Predict on the unchanged official testing CSV.
- ☐ Save `results/p3.csv` in the shared format.

Week 3

- ☐ Record accuracy, precision, recall, and F1-score.
- ☐ Write 3-5 sentences about the results and how long training took.
- ☐ Make sure your script and README let someone rerun your model.

Submit: `person3_svm/train.py`, `person3_svm/README.md`, short results summary, and `results/p3.csv`.

Person 4 - Isolation Forest Model

In plain English: Instead of learning specific attack labels, your model learns what normal traffic looks like and flags unusual traffic as suspicious.

Week 1

- ☐ Download both CSVs and set up `person4_isolation_forest/train.py`.
- ☐ From the **training CSV only**, select rows with `label = 0` (normal traffic).
- ☐ Prepare numeric model inputs; try a small sample first if needed.

Week 2

- ☐ Train scikit-learn's `IsolationForest` on **normal training rows**.
- ☐ Run it on all test rows. Its output is `1 = normal`, `-1 = unusual`; convert that to our format: **normal = 0, unusual/attack = 1**.
- ☐ Save `results/p4.csv` with the original test row order.

Week 3

- ☐ Record accuracy, precision, recall, and F1-score against the real test labels.
- ☐ Write 3-5 sentences on whether unusual traffic matched known attacks.
- ☐ Explain that detecting *anomalies* is different from identifying *known attacks*.

Submit: `person4_isolation_forest/train.py`, `person4_isolation_forest/README.md`, short results summary, and `results/p4.csv`.

Person 5 - Dashboard + Comparison

In plain English: Make one place where the class can see what the four models predicted and which performed best. You do **not** need to train anyone else's model.

Week 1

- ☐ Create a basic **Streamlit** page with the project title and short explanation.
- ☐ Make two tiny **fake example CSVs** with the required columns to test your page. Do not wait for real model files.
- ☐ Show a table of predictions from the example CSVs.

Week 2

- ☐ Add accuracy, precision, recall, and F1-score calculations from the CSVs.
- ☐ Add a simple bar chart comparing model scores, and optionally a confusion matrix.
- ☐ Add a friendly message if a results file is not available yet (instead of crashing).

Week 3

- ☐ Replace the example data with `results/p1.csv` through `results/p4.csv` as they arrive.
- ☐ Check that every file has the **same row IDs and true labels** before comparing scores.
- ☐ Prepare one screenshot and a short demo for the final presentation.

Submit: `person5_dashboard/app.py`, `person5_dashboard/README.md`, a screenshot, and a short final comparison.

Simple 3-week timeline

When	What should be finished?	Does anyone need to wait?
End of Week 1	Everyone has the dataset and a working starting script or dashboard.	**No.** Person 5 uses fake example files.
End of Week 2	People 1-4 have working models and can create prediction CSVs; Person 5 can read sample CSVs and show metrics.	**No.** Everyone tests their part separately.
End of Week 3	All four CSVs are collected, dashboard displays results, and each person contributes a short explanation.	**Only final assembly.** Missing models can be added later.

Final checklist for the whole group

- ☐ Each person uploads their **own** code, README, and required results.
- ☐ All available model CSVs contain `row_id,y_true,y_pred` for the **same** test rows.
- ☐ The dashboard displays metrics and handles a missing model file gracefully.
- ☐ The presentation explains the **problem, dataset, four methods, comparison, and limitations**.
- ☐ Each member can explain what **their own** component does in 20-30 seconds.

If you are catching up: Read the first page, find your person number, and start at **Week 1**. You do not need to understand or run anyone else's code to begin. Your finished contribution is the files listed under **Submit**.

Dataset source: University of New South Wales, UNSW-NB15 official dataset. The dataset's original authors request citation in academic work; include their citation in the final report or slides.